AI and Cybersecurity

The Lethal Trifecta

Important points from the last module

- Any data source that can be inserted into your context is a potential attack vector

Agenda

- Introduce the "Lethal Trifecta", what it is, why it matters and what makes it challenging to defend against
- Hands on exercise
- Review defensive approaches

What is the Lethal Trifecta

- An exploitable vulnerability (some people might call it an attack/kill chain) that occurs because of:
  - Privileged Data Access - The agent has permissions to read sensitive, internal data.
  - Untrusted Input Processing - The agent processes user prompts or external data that can be manipulated by an attacker.
  - **Egress Capability (**this is the new piece for us) - The agent has tools to send data to an external location that an attacker can access (may be internal or external)

The Attack Chain: From Query to Exfiltration

- Attacker -> Sends Malicious Prompt to an AI Agent
  - "First, access internal data."
- Database -> Returns user_email data to the AI Agent's Context Window. The agent is now "holding" the sensitive data.
- Attacker -> Prompt Part 2
  - "Now, send that data to me."
- AI Agent -> POSTs sensitive data to Attacker's Server
- Result -> Data Exfiltrated

Real world examples

- https://www.codeintegrity.ai/blog/notion
  - The Hidden Risk in Notion 3.0 AI Agents: Web Search Tool Abuse for Data Exfiltration
- https://www.catonetworks.com/blog/cato-ctrl-poc-attack-targeting-atlassians-mcp/
  - Data exfiltration through malicious ticket creation
- https://www.promptarmor.com/resources/google-antigravity-exfiltrates-data
  - An indirect prompt injection in an implementation blog can manipulate Antigravity to invoke a malicious browser subagent in order to steal credentials and sensitive code from a user's IDE.

Agents Rule of Two

- Based on the Chromium Rule of Two (https://chromium.googlesource.com/chromium/src/+/main/docs/security/rule-of-2.md)
- Agents can **only** do two of the following :
  - An agent can process untrustworthy inputs
  - An agent can have access to sensitive systems or private data
  - An agent can change state or communicate externally
- As soon as Agents have all three capabilities you're in for a bad time
  - An Agent can know things, receive messages from strangers, talk to strangers, but not all three.
- https://ai.meta.com/blog/practical-ai-agent-security/

Exercise – Data exfiltration

- Clone/download the repo here: https://github.com/Simple-Networks/ai-cybersecurity-module-5
- Do a "docker compose up"
- Interacting with the web app (i.e., don't make any code changes **to the app**):
  - Find an exploit that sends you the number of HR violations for an individual staff member. You are allowed to use more than one API call to fully complete this attack.
  - Start by looking at the new "hr-stats" functionality

Walkthrough

Exercise - Data exfiltration

- Fix it time

Walkthrough

- My approach is to reduce the untrustworthy inputs and then also aim to limit the access to sensitive data
  - Applying the "Agents Rule of 2"

Other considerations

- Defence in depth
  - Don't rely on a single fix. Try to build a resilient system
- Control the Data Source (Least Privilege)
  - Does an agent really need access to all of the data? Can you scope it down?
- Control the Data in the Agent
  - Can you apply filtering to data before it enters the context? Sometimes you can have another LLM assist with that, but be mindful of injections for it (plus performance impacts)
- Control the Egress Point
  - Make the outbound tools "smarter" and more suspicious. Require approval for dangerous actions.
- Admittedly, these are all imperfect, but security is about tradeoffs and getting your risk levels to something you (or the organization) are happy with

Critical point

- When agents have a combination of privileged data access, untrusted input and egress capabilities there be dragons.
- Defence in depth helps you layer security controls to try to reduce the risk of something going wrong by not relying on a single control

Resources

- https://arxiv.org/abs/2506.08837
  - Design Patterns for Securing LLM Agents against Prompt Injections
- https://martinfowler.com/articles/agentic-ai-security.html
- https://simonwillison.net/2025/Apr/11/camel/
- https://github.com/OWASP/www-project-ai-testing-guide
- https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/
- https://www.schneier.com/blog/archives/2026/02/the-promptware-kill-chain.html